PUShare: Course Materials for NewUU Students in One Place

SE202 Mobile Programming · Midterm Project · Team Strangers

Abduraxmon Qutbiddinov (240426), team lead: user flow & screens · Umar Toshqulov (230070): problem research
Adxamjon Raimov (240422): presentation lead · Oyatillo Axadjonov (240446): data model

1. Problem statement and target users

Owned by: Umar Toshqulov

NewUU students often struggle to find course materials such as lecture slides, notes, practice exercises, and past exam papers. These resources are scattered across Telegram groups, university platforms, and shared folders, with no consistent organization. As a result, students spend extra time searching for files, asking classmates, or checking multiple sources. Finding materials from previous semesters or specific academic years is particularly difficult.

Core problem: the lack of a centralized, structured repository for course materials makes resource discovery inefficient and time-consuming.

Target users: NewUU students across all majors and years of study. Our survey evidence so far comes mostly from third-year Software Engineering students, who are our first users.

2. Evidence: survey report

Owned by: Umar Toshqulov

We ran an online survey among NewUU students between October 21 and 24, 2026. It collected **19 responses** and examined how students access academic materials, the difficulties they experience, the time needed to find resources, and whether these difficulties affect their assessment preparation. Questions and charts are in the Appendix.

Indicator	Result
Students using Telegram for academic resources	100% (19/19)
Students who experienced difficulty finding materials	94.7% (18/19)
Students who missed materials before quizzes or exams	63.2% (12/19)
Students experiencing difficulties daily or weekly	57.9% (11/19)
Students relying on classmates for materials	47.4% (9/19)
Students who spent over 30 minutes finding a resource	26.3% (5/19)

Analysis. All 19 respondents use Telegram for academic materials, and many also rely on classmates, Hero LMS and course websites. 63.2% have been unable to find a required material in time before a quiz or exam at least once. This is preliminary evidence that finding materials is a meaningful problem, which PUShare addresses with one centralized app for finding and sharing course-specific resources.

Limitations. 19 responses is above the required 10, but still a small sample. Most respondents were third-year Software Engineering students, so the results are not representative of all NewUU students, and the benefits of PUShare would need to be validated through further testing.

3. Solution approach

App concept

Owned by: Adxamjon Raimov

PUShare is a mobile app where NewUU students find and share study materials for their own major. A student signs up with their @newuu.uz email, and the home screen then shows only the accepted materials for their major. They can filter by academic year, study year and course, because courses and their order can differ even inside one major. Students can also upload notes, links, images and PDFs, which an admin reviews before they appear for everyone. Major, year and email are fixed after sign-up, and a student who needs a change sends a request to an admin from the Account screen. **The single most important thing a user can do is find the material they need for a course in a few taps, without asking around in group chats.**

Main user flow

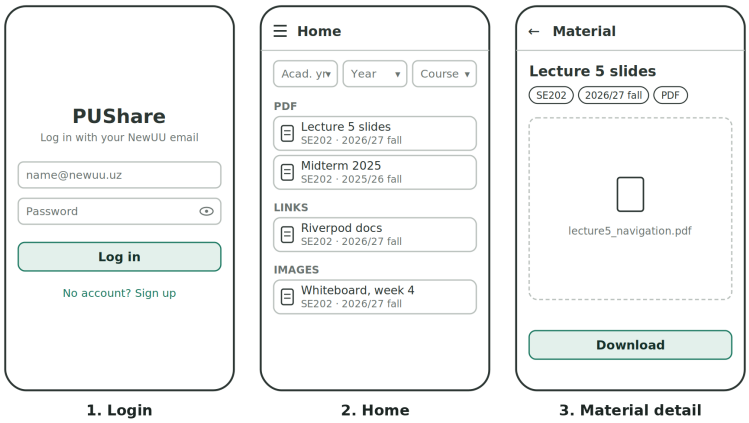
Owned by: Abduraxmon Qutbiddinov

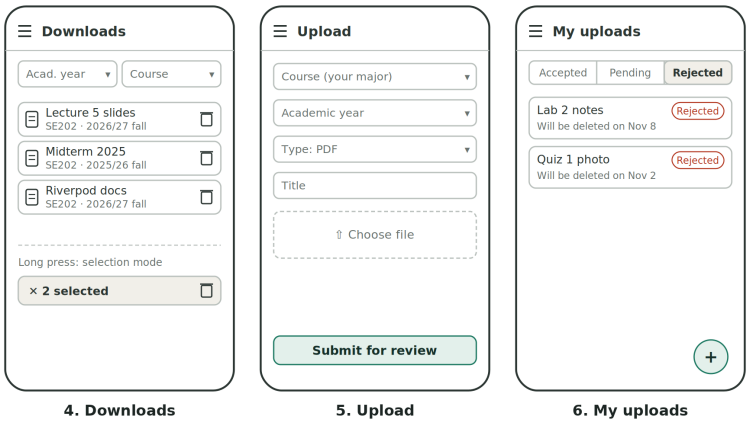
Step	Screen
1. Log in with a NewUU email	Login
2. Filter by academic year, study year, course	Home
3. Open a material	Material detail
4. Download it, reopen later without searching	Downloads
Upload path: share a material, which goes to pending for admin review	Upload
See whether your upload was accepted or rejected	My uploads

Screens and wireframes

Owned by: Abduraxmon Qutbiddinov

Six core screens cover the main user flow, each with a low-fidelity wireframe below. **1. Login:** lets a student in with their @newuu.uz email. **2. Home:** accepted materials for the student's major, filtered by academic year, study year and course. **3. Material detail:** shows one material and lets the student download it. **4. Downloads:** reopen saved materials without searching again; includes selection mode for deleting several. **5. Upload:** share a material for a course in the student's major. **6. My uploads:** shows the review status of everything the student has uploaded.





Shared navigation: a side drawer with Home, Upload, My uploads, Downloads and Account appears on all main screens. **Supporting screens** (no separate wireframe): Sign up (email, major, study year, password); Confirmation code (one 6-digit field, for sign-up and password change); Account (email, major and year read-only, links to Edit info, Change password, Delete account); Edit info and Change password (single-purpose forms). Email can't be changed; major and year only through an Edit info request to an admin.

4. Flutter plan

Owned by: Oyatillo Axadjonov · This is our planned approach; details may change when we build the app.

Packages and project structure

Besides core Flutter we use two packages: `flutter_riverpod` for state and `go_router` for navigation. We have no backend yet, so the app reads from repository classes that return fake in-memory data after a short `Future.delayed`. Screens call them with async/await. When a real backend exists, we swap the repository implementation and the screens stay the same. The lib folder: `main.dart`, `app/` (router and theme), `models/`, `data/` (repositories), `providers/`, `screens/` (one folder per screen) and `widgets/` (drawer, material card). `main.dart` wraps the app in a `ProviderScope`, and `MyApp` returns `MaterialApp.router`.

Navigation

Route	Screen	Route	Screen
/login	Login	/downloads	Downloads
/signup	Sign up	/account	Account
/confirm?purpose=signup|changePassword	Confirmation code (both flows)	/materials/:id	Material detail
/	Home	/account/edit-info	Request to change major/year
/upload	Upload	/account/change-password	Change password
/my-uploads	My uploads		

The five drawer destinations (Home, Upload, My uploads, Downloads, Account) are ordinary routes. Each has its own `Scaffold` with `drawer: const AppDrawer()`, so the hamburger icon appears in its AppBar; `AppDrawer` is a shared stateless widget, defined once and reused. Drawer items close the drawer and call `context.go`, which replaces the stack. Material detail and the account sub-forms are opened with `context.push`, so back returns to the previous screen.

Drawer items use `go` on purpose: a drawer switches between top-level sections, so there should be no back history between them. The tradeoff is that the previous section's local state (e.g. scroll position) is lost; keeping it would need `StatefulShellRoute`, which is out of scope. Material detail only receives the `:id` from the path and looks the material up through a provider, so we never pass a whole object between screens and there is one source of truth. The confirmation screen is opened with `context.push<bool>` and closes with `context.pop(true)` on success; after sign-up the caller goes Home, after a password change it goes back to Account. An auth guard in `GoRouter.redirect` sends logged-out users to /login and keeps logged-in users away from /login and /signup.

State management

We use Riverpod. State that more than one screen needs goes into a provider; state that only one screen needs stays in the widget. In `build` we use `ref.watch`, in callbacks (button presses) `ref.read`.

State	Where it lives	Reason
Current user	`authProvider` (AsyncNotifier<User?>)	Used by the router guard, the drawer and the upload screen
All materials	`allMaterialsProvider` (FutureProvider)	Single source of truth
Home filters	`homeFiltersProvider` (Notifier<MaterialFilters>)	Must survive opening a detail page and coming back
Home list	`homeMaterialsProvider` (derived)	Watches materials, filters and user; keeps accepted materials of the user's major
One material	`materialByIdProvider` (.family)	Used by the detail screen
Downloads	`downloadsProvider` (Notifier<List<DownloadRecord>>)	Shared by Downloads and the detail screen
Is downloaded	`isDownloadedProvider` (.family, derived from downloadsProvider)	Decides whether the detail screen shows "Download" or "Delete from downloads"
My uploads per status	`myUploadsProvider` (.family by status)	One list per tab
Form fields, controllers, dropdown values, "submitting" flag, multi-select set	Local widget state (setState)	Only one screen uses them and they are thrown away on exit

Forms and validation

Every form uses `Form`, a `GlobalKey<FormState>` and `TextFormField` (or `DropdownButtonFormField`). On submit we call `_formKey.currentState!.validate()` first. Controllers are created in `initState` and disposed in `dispose`.

Field	Rule	Field	Rule
Email	Not empty, ends with @newuu.uz	Confirmation code	Exactly 6 digits
Password	At least 8 characters	File	Must be an allowed type
Confirm password	Same as the new password field	Link	Valid http or https URL
Major, year, course, academic year, type	Must be selected	Text material	Not empty

Screens (widgets and navigation)

- **Login.** A `Scaffold` with a padded `Column`: email field, password field (obscured, with an eye `IconButton`), a login button and a `TextButton` to sign-up. Controllers are local and the button calls the login method on `authProvider`.
- **Sign up.** A `SingleChildScrollView` around a `Form` and a `Column`, so the keyboard does not cause an overflow: email, major dropdown, year dropdown (1 to 4), password, submit. On submit it validates, awaits the sign-up call, pushes the confirmation screen and goes Home if the result is true. Major and year can only be set here; email is never changeable.
- **Home.** `Scaffold` with the `AppDrawer`, and a `Column` with a filter `Row` and an `Expanded` list. The Row has three `Expanded` dropdowns (academic year, study year, course). Materials are grouped by type: to build this with a single `ListView.builder`, we flatten the grouped data into one list where an item is either a type name (`String`) or a `StudyMaterial`; the builder returns a header `Text` or a `MaterialCard`. Changing a filter updates `homeFiltersProvider`, which rebuilds the list. Tapping a card calls `context.push('/materials/<id>')`. An empty result shows an empty-state widget.

- **Material detail.** `Scaffold` with an `AppBar` and a `ListView`: title, a `Wrap` of `Chips` (course, academic year, type) and the content, which depends on the type (`Image.network` for images, `Text` for text, link text for links, file icon and name for PDFs). The bottom button says "Download" or "Delete from downloads", based on `isDownloadedProvider(id)`.
- **Upload.** A `Form` inside a `ListView`: course dropdown (only courses of the user's own major, the upload constraint), academic year, type, title, and a content field that changes with the type (URL field, multiline text field, or file picker button). On submit we set `_submitting = true` to disable the button, await `repo.add(...)` with status pending, check `mounted`, then call `context.go('/my-uploads')`.
- **My uploads.** A `Column` with a `SegmentedButton` (Accepted, Pending, Rejected) above an `Expanded(ListView.builder)` of cards; a `FloatingActionButton` opens Upload. Rejected cards show a chip and "Will be deleted from the server on <date>". Multi-select and delete work as on Downloads, through one shared selectable list widget.
- **Downloads.** Similar view and behavior to Home, grouped by type with the same grouping function. Each card has a delete icon, and a long press selects several cards to delete at once.
- **Edit info.** Major and year cannot be changed in the app, so this screen is a request form: a dropdown for what to change (major or study year), a second dropdown for the new value (its options depend on the first choice), and an optional multiline note for the admin. On submit it validates, sets `_submitting = true`, awaits `repo.sendInfoChangeRequest(field, newValue, note)`, checks `mounted`, shows a `SnackBar` ("Request sent to admin") and calls `context.pop()`. The backend is responsible for emailing the admin; until it exists, the fake repository only simulates the delay.
- **Confirmation code.** A `Column` with an explanation text, one 6-digit field, a Confirm button and a Resend `TextButton`. The `purpose` query parameter decides which text is shown and which repository call is made.
- **Account.** A `ListView` of `ListTiles`. Email, major and year are shown read-only. Below them are Edit info, Change password and Delete account. Edit info and Change password use `context.push`. Delete account first shows an `AlertDialog` to confirm, then calls the delete method on `authProvider`, and the router guard sends the user to /login.
- **Change password.** A `Form` with current password, new password and confirm new password. On submit it validates, awaits the change request, pushes the confirmation screen with `push<bool>`, and goes back to Account if the result is true.
- **Shared widgets.** `AppDrawer` is a `Drawer` with a `ListView` of five `ListTiles`. It highlights the current item by comparing the current route location with each item's path.

5. Data model

Owned by: Oyatillo Axadjonov

Four classes and two enums. The class is called `StudyMaterial` instead of `Material`, because `Material` is already a Flutter widget and the two names clash when material.dart is imported.

```
enum MaterialStatus { pending, accepted, rejected }
enum MaterialType { text, image, link, pdf }

class User {
  final String email;
  final String major;
  final int studyYear;
  final DateTime createdAt;
  const User({required this.email,
    required this.major, required this.studyYear,
    required this.createdAt});
}

class Course {
  final String id;
  final String name;
  final String major;
  final int studyYear;
  final String academicYear; // e.g. "2026/2027 fall"
  final DateTime createdAt;
  const Course({required this.id, required this.name,
    required this.major, required this.studyYear,
    required this.academicYear, required this.createdAt});
}

class DownloadRecord {
  final String materialId;
  final DateTime downloadedAt;
  final String localPath;
  const DownloadRecord({required this.materialId,
    required this.downloadedAt,
    required this.localPath});
}
```

```
class StudyMaterial {
  final String id;
  final String courseId;
  final String major;
  final String academicYear; // e.g. "2026/2027 fall"
  final String title;
  final MaterialType type;
  final String content;      // text body, URL, or file name
  final String? description;
  final MaterialStatus status;
  final String uploadedByUserEmail;
  final DateTime uploadedAt;
  final DateTime? reviewedAt;

  const StudyMaterial({
    required this.id, required this.courseId,
    required this.major, required this.academicYear,
    required this.title, required this.type,
    required this.content, this.description,
    required this.status,
    required this.uploadedByUserEmail,
    required this.uploadedAt, this.reviewedAt,
  });

  // When a rejected material is removed from the server.
  DateTime? get scheduledDeletion =>
      status == MaterialStatus.rejected && reviewedAt != null
          ? reviewedAt!.add(const Duration(days: 14))
          : null;
}
```

How the fields map to the screens:

- Sign up and Account show `User.email`, `major` and `studyYear`.
- Home filters use `StudyMaterial.academicYear`, `Course.studyYear` and the course name (through `courseId`). Lists group by type and show `title` and `academicYear`.
- Material detail shows title, type, content, the course name and academicYear. Upload fills `courseId`, academicYear, type, title and content, and sets status to pending.
- My uploads sorts by status and shows `scheduledDeletion` on rejected items. Downloads reads `DownloadRecord`, joined to `StudyMaterial` by `materialId`.
- Filter values live in a small UI-only class, `MaterialFilters { String? academicYear; int? studyYear; String? courseId; }`, which is not stored anywhere.

6. Out of scope

Owned by: Adxamjon Raimov

The app we build for the course covers the screens, navigation, state and forms. Everything below is needed for a real product, but we haven't learned it yet, so the prototype fakes it or leaves it out.

- **Backend and database.** Accounts, materials and courses live in fake in-memory repositories; a real server would store them and keep them in sync between users.
- **Real authentication and email codes.** The 6-digit confirmation code is simulated; sending it to an @newuu.uz inbox and checking it needs a server.
- **Admin review panel.** Pending, accepted and rejected statuses exist in the app, but the admin who changes them does not; statuses are preset fake data.
- **Messages to the admin.** The Edit info form only validates and shows a "request sent" confirmation; delivering the message needs a backend.
- **File storage and real downloads.** Storing PDFs and images on a server, saving them to the phone, and periodically cleaning up rejected files needs file storage and background tasks.
- **Push notifications.** Telling a student "your upload was accepted/rejected" would need a notification service.
- **Keeping screen state between drawer sections.** `context.go` replaces the stack, so local state such as scroll position is lost; keeping it would need `StatefulShellRoute`.
- **Community voting and AI moderation.** Students voting on uploads and a small AI model that normalizes course info and checks for rogue uploads is a later enhancement.

7. Who did what

- **Abduraxmon Qutbiddinov (team lead):** main user flow, screens and wireframes; designed the survey and distributed it together with Oyatillo; coordinated the team, made final consistency fixes across sections, assembled and submitted the document.
- **Umar Toshqulov:** problem statement and target users; wrote the survey report (findings, analysis and limitations).
- **Adxamjon Raimov:** app concept and the out-of-scope list; prepared and rehearsed the presentation.
- **Oyatillo Axadjonov:** Flutter plan (packages, project structure, navigation, state management, forms and screen layouts) and the data model; distributed the survey together with Abduraxmon.

8. AI usage details

Oyatillo created the first draft of the document with the help of AI: rough answers to every section, based on our ideas, the survey and the planned structure, as far as they were known at that point. Each member then took that document, which served as our shared knowledge base and template, and used LLMs, including Claude (Anthropic), to design and write their own part. We also used Claude to help structure the document, improve wording, check the Flutter plan and data model for errors, draw the low-fidelity wireframes from our screen descriptions, and build the presentation slides.

Appendix: Survey evidence

Online Google Form on how NewUU students find course materials, October 21–24, 2026, 19 responses.

Questions: (1) Your year: 1 / 2 / 3 / 4. (2) Your major: list of NewUU majors. (3) Where do you usually get course materials from (pick all that apply)? Telegram / Hero LMS / instructor by email / from friends or other students / paper copies / course websites. (4) How often did you have trouble finding materials you needed? Daily / Weekly / Monthly / Occasionally / Never. (5) Last time you looked for a specific material, how long did it take you to find it? Under 2 min / 2–10 min / 10–30 min / over 30 min / never found it. (6) Have you ever missed a material before a quiz or exam because you couldn't find it in time? Yes, once / Yes, more than once / No.

Results (Google Forms response summaries):

